

Flashcard Library App – Product Case Study

Overview

The Flashcard Library App is intended for people who wish to review information in flash card format with a question on one side and answer on the other. It includes an ability to import text or CSV files, as well as enter and edit question/answer pairs directly into the app.

I took this from concept to product release independently, covering product development, architecture, development, QA testing, and documentation.

The Problem

Observed Gap

I had the idea for a flashcard library app where a user would be able to easily load new flashcard decks at will, particularly while traveling. The need arose from a desire to tour Europe and learn some of each language as I visit each country.

Why Existing Tools Fell Short

I did some looking at the Google Play marketplace in 2023, but didn't see an existing app that didn't include requiring permissions or logins, or the apps were cluttered with unneeded features. There were also privacy concerns when third party ads were included, or when the app required more personal user information.

The Core User Need

The use case at the core of this app is the perspective of a person touring multiple foreign countries, where the user wants to learn some of the language of the country they are about to visit. This app enables them to load vocabulary into the app and immediately be able to review them in flashcard format.

The Solution

What the App Does

This app removes all of the bloat and focuses on being a tool to load flashcard decks to review as needed.

- The user either loads a file (plain text or CSV) or uses an interface to enter questions and answers, and the deck is created.
- The user clicks on a flashcard deck to start reviewing cards.
- Once in a card deck, touching the card flips between the question and answer sides of the card.
- Navigation in Review mode allows moving between cards, as well as re-shuffling the deck and starting over.
- There's an option to swap the question and answer sides of cards while reviewing. In particular, for language learning you can then start either with the word in your native language, or start with the word in foreign language.

Key Design Decisions

- **File-based input**
 - Allow users to prepare lists in any file editor they feel comfortable with, and support simple question-and-answer structures for text and CSV files.
- **Supported formats (CSV, plain text, in-app text box)**
 - Text was the initial import format, having questions on odd lines and answers on even lines.
 - CSV import was added for use with spreadsheets, where the questions are in column 1 and answers in column 2.
 - UI input was added for people that didn't have access or time to make a separate file and knew what they wanted to put in a flashcard deck.
- **No accounts / no backend**
 - Minimizing functionality like keeping user accounts reduced a lot of overhead to finish the app.
 - Lack of collection of any user data is a centerpiece for privacy and trust.
 - Local storage for flashcard decks also enforces privacy.
- **Progressive Web Application (PWA) over native app**

- This app actually started as a native app, which I learned requires an LLC to submit to an app store without disclosing my name or personal address. It also requires upgrading and re-testing when each OS is updated.
 - I pivoted to using a PWA that utilizes web technology and a fixed set of code versions that I control, which means far less upkeep cost.
 - It allows users to still download the app (in a sense), but allows access without cost.
 - Access to the PWA app does not require an account on the Apple App Store or Google Play; it is accessible from any web browser.
-

Current State

What Works in PWA Version

- File import (CSV, plain text) – Tested and working
- Flashcard review mode – All functionality tested and working
- Day/Night mode - Tested and working
- In-app Help Icon - Tested and working
- Adding question to deck - Tested and working
- Slide to delete card or deck - Tested and working
- Main Menu - Tested and working

Known Limitations

- Text inputs outside of the "add question" screen are not working in PWA format, expected to be fixed soon.
 - There are some minor UI issues, including not-quite centered elements.
 - Exporting the deck library generates an error.
-

Product Roadmap

Future Features

- **Quiz Mode**
 - User must select right answer from four given answers

- Answers are generated from the deck
 - At least one wrong answer is very similar to the right answer, which makes the user think about which one is right
 - **Audio / Hands-Free Mode**
 - Text-to-speech card reading for review while commuting, driving, or otherwise reviewing content without looking at the phone
 - Review mode: question is read, five seconds elapse, answer is read
 - Quiz mode: only question is read, with a ten second pause to answer
 - Pauses would be configurable
 - **Disable cards in Review Mode**
 - When in Review Mode reviewing cards and the user is trying to learn a deck, cards may end up being well known. A feature could be added to let the user disable cards while reviewing so that future reviews will not include that card.
-

Reflections

What I Learned

I intentionally started and finished this app utilizing Expo and React Native with the original intent to release it to both the Apple App Store and Google Play. I learned a lot about React Native and its libraries in putting together a solid user experience with the features I wanted to provide for users.

I was able to exercise judgment on whether to go deeper on a feature or move on to the next item. I kept a to-do list of items in front of me, and if something seemed like it would slow the timeline too much, I backed it out and put it on a "future functionality" list.

When I got to the point where I could release it on the app stores, I very belatedly realized that I would have to keep the app current with new OS releases and test it each time. I recognize the duty as important, but as I was still the only person working on it and didn't have plans to expand, I decided it was not a road I wanted to take. Using a PWA allows me to set the software versions, and it can live in a software ecosystem that rarely requires updating and re-testing.

What I'd Do Differently

Utilizing AI assisted coding would have greatly sped up development time at the cost of more intimate knowledge of React Native development. It would also require extensive code review rather than coding. However, the speed and ability to add more features would have been a nice bonus.

The very late realization of overhead for adding apps to the app stores and the upkeep plus LLC requirement was something that needed to be scoped earlier in the project. In future projects, I would look clearly at where I wanted the final product to land, and what would be required when getting there, as well as doing occasional confirmations that the requirements had not changed.

Links

- **Live App:** *coming soon*
- **Source Code:** <https://github.com/opensorcerer20/quizapp>
- **Contact:** opensorcerer20@gmail.com